Final Report PSC INF12

Synergistic Reasoning in Multi-Agent Systems

Synergistic network anomaly detection with a multi-agent system on knowledge graphs – application to telecommunication networks

Students	DES BOSCS Raphaël, CHICHE Paul, SIGNORETTI Andrea, QILE Liu
Tutor	TAILHARDAT Lionel, AI Researcher at Orange
Coordinator	LUTZEYER Johannes, Assistant Professor, LIX

Contents

1 Introduction **3**

1.1 Context and motivations . . . 3
1.2 State of the art . . . 3
1.3 Problem statement . . . 5
1.4 Project objectives . . . 5
1.5 Main achievements . . . 6
1.6 Report outline . . . 6

2 Intermediate results and key lessons **6**

2.1 Strategic pivot: from the objective function to a working system . . . 6
2.2 Learning JaCaMo through implementation . . . 7
2.3 From isolated cases to ontology-aligned families . . . 7
2.4 Importance of the *a priori* / *a posteriori* distinction . . . 7
2.5 Designing datasets as a scientific instrument . . . 8
2.6 Lessons retained for the final architecture . . . 8

3 Architecture of the Multi-Agent System **8**

3.1 Design principles . . . 8
3.2 Overview: a three-level pipeline . . . 9
3.3 The JaCaMo platform . . . 10
3.4 The agent–environment interface: PythonExecArtifact . . . 10

4 Level 1: Detection Agents by NORIA-O Facet **11**

4.1 General organization: a priori and a posteriori . . . 11
4.2 Structural family . . . 12
4.3 Dynamic family . . . 13
4.4 Functional family . . . 14
4.5 Procedural family . . . 14

5 Level 2: Conditional Reasoning Agents **15**

5.1 Motivation and correlation principle . . . 15
5.2 Common framework `common.py` and execution principle . . . 15

5.3 Catalogue of implemented diagnosers 16
5.4 Approximating the objective function: scoring the diagnoses 18

6 Level 3: Narrative Synthesis by an LLM **19**
6.1 Separating decision and communication 19
6.2 Prompt construction and model 20
6.3 Example output on the noria-0.2 scenario 20

7 Experiments and Results **21**
7.1 Tested dataset catalogue and experimental processes 21
7.2 First architectural validations on NORIA-O 0.2 22
7.3 Various evaluations on the synthetic catalogue 23
7.4 Ablation study and comparison with LLM detector and diagnoser 25

8 Conclusion **27**
8.1 Answer to the research question 27
8.2 Results and empirical answers to the research question 28
8.3 Discussion, limitations and perspectives 29

9 References **29**

1 Introduction

1.1 Context and motivations

Telecommunication networks, or more broadly ICT infrastructures (*Information and Communication Technologies*), are today critical systems on which many essential services depend. Their supervision raises a well-known difficulty: anomalies are rarely confined to a single technical layer. They may result from a structural weakness, a software change, a hidden functional dependency or a procedural breakdown in the operational chain. Detecting and interpreting these anomalies therefore requires the ability to reason over heterogeneous data, at several scales, and with strong explainability requirements.

To handle this complexity, Orange relies on a digital twin represented as a Knowledge Graph. Inherited from the Semantic Web, this representation links heterogeneous sources in a unified form as RDF triples. In our context, the semantics of the graph is structured by the **NORIA-O** ontology, which organizes the description of the network around four complementary facets: *structural*, *dynamic*, *functional* and *procedural*. This structure is not merely descriptive: it opens the way to distributed reasoning, in which specialized agents query the graph and produce partial diagnoses that are then correlated at a higher level.

This PSC is part of this perspective. It aims to design a multi-agent system able to detect and interpret anomalies in a knowledge graph representing a telecommunication network, while maintaining a high level of explainability. This point is central: in a field as sensitive as network operations, the performance of a diagnostic system cannot be assessed solely by its raw accuracy; it must also be judged by its ability to justify its conclusions and to make the reasoning chain that produces them auditable.

1.2 State of the art

Our state-of-the-art analysis, carried out in the first phase and consolidated in the second, confirmed that the combination of knowledge graphs, the NORIA-O ontology and multi-agent systems is a relevant theoretical framework for maximizing explainability and operational efficiency at the same time. It is organized around four axes, which we present in turn.

1.2.1 Semantic representation and knowledge graphs

ICT network management faces unprecedented challenges — diversity of services, density of the infrastructure and volume of heterogeneous data — that make the mere collection of raw data insufficient. Operators now rely on *autonomous digital twins* [6], updated in real time, which provide a structured representation while allowing analysis by agents.

Knowledge graphs (KG) [3] are the natural choice in this context. Derived from the Semantic Web, they rely on the *Resource Description Framework* (RDF) model, in which information is structured as (subject, predicate, object) triples forming a graph with semantically typed edges. This format gives KGs a high degree of *explainability*, *modularity* and *flexibility* in the face of heterogeneity. Built on W3C standards [16], this representation allows SPARQL to be used to query the data and identify complex patterns, while OWL makes it possible to define ontologies and derive automatic reasoning from them.

An *ontology* is a semantic network defining the concepts of a domain and their relations. In the ICT domain, it can establish that a router is a type of network equipment, that two connections are redundant, or that a critical incident impacts emergency services. We specifically use **NORIA-O** [13], developed at Orange, which structures information along four facets — structural, functional, dynamic and procedural. NORIA-O stands out from the traditional information models of the field (TM Forum SID, ITU-T M.3100, TOSCA) through its richer semantics and its explicit focus on incident management. The associated thesis by Tailhardat [12] details the construction

of this digital twin usable by agents. In practice, these models run on RDF graph databases such as *OpenLink Virtuoso* (large volumes) or *GraphDB* (automatic inference); our choice of Virtuoso is discussed in Section 3.

Finally, task optimization on KGs (representation learning, reasoning, completion) relies on a set of classical *objective functions* [15]. Their evaluation has several dimensions — adaptability, effectiveness, cross-scenario compatibility and interpretability — the latter being central for us. As detailed in Section 2.1, the formalization of this objective function to drive agent scheduling was recalibrated during the project: in the present architecture, the coordinator's severity hierarchy is a first pragmatic approximation of it.

1.2.2 Multi-agent systems: classical and agentic paradigms

Once the digital twin is operational, the processes acting on its data must be defined to make it *autonomous*. This is done with *agents*, software entities able to perceive their environment, reason and act. In network supervision, some specialize in collection, others in detection, others in planning corrective actions. This typology comes with an *explainability–accuracy trade-off* : *deterministic* agents offer full explainability at the cost of high complexity, while *probabilistic* ones — based on *process mining* [14] or on graph embeddings [15] — achieve better accuracy but lose traceability. The article by Hussain et al. [4] provided the architectural basis of our solution by identifying the necessary coordination mechanisms; the challenge is then to *order* and *prioritize* the agents to obtain a trade-off between their strengths.

Our implementation relies on the BDI (*Belief-Desire-Intention*) paradigm of Rao and Georgeff [9], the canonical formalization of a rational agent that updates its *beliefs* through perception, deliberates between *desires* (candidate goals) and commits to an *intention* (action plan). This model is implemented by AgentSpeak/Jason [2] within the **JaCaMo** [5] platform, which combines three complementary components: *Jason* for the BDI logic, *CArtAgO* for shared environment artifacts, and *Moise* for formalizing the organization (roles, norms, missions). This threefold dimension (agents – environment – organization) structures our architecture (Section 3).

We position this choice with respect to the recent wave of **LLM-based agentic frameworks** (AutoGen [17], CrewAI, LangGraph), in which agents are LLM instances coordinated through natural-language exchanges. These frameworks offer remarkable flexibility but have two prohibitive limitations for our use case: (i) decisions rely on probabilistic, non-auditable LLM inferences, which is incompatible with the explainability requirements of the ICT domain; (ii) inter-agent coordination relies on natural-language prompts rather than formal semantics, ruling out any guarantee on the system's behavior. The choice of JaCaMo is therefore deliberate: it favors the formal traceability of BDI over the flexibility of LLM exchanges, at the cost of lower expressiveness.

1.2.3 Integrating LLMs and knowledge graphs

The integration of language models and knowledge graphs became a research field in its own right between 2023 and 2025, for which Pan et al. [8] propose a reference taxonomy. Three paradigms stand out. In the *KG-enhanced LLM* approach, the graph enriches the LLM's context (retrieval-augmented generation over a KG) to reduce hallucinations. In the *LLM-augmented KG* approach, the LLM helps build, complete or correct the graph. Finally, the *synergized reasoning* approach, illustrated by Think-on-Graph [11], has the LLM traverse the graph iteratively to reason step by step.

Our Level 3 sits in a fourth regime, more restrictive than the previous three: that of the **strictly verbalizing LLM**. In this regime, the diagnosis is entirely established by upstream symbolic components (L1 workers + L2 coordinator), and the LLM only rephrases into prose a structured report that is already complete. This choice differs markedly from *LLM-as-reasoner* approaches: it deliberately sacrifices the LLM's reasoning expressiveness to preserve full traceability of the decision, a property essential to the targeted operational setting. This architectural choice is discussed in detail in Section 6.

1.2.4 Explainability: the XAI framework

Since our project is centered on explainability, we place it within the field of *explainable AI* (XAI). The reference taxonomy of Arrieta et al. [1] distinguishes two main families: *intrinsically interpretable* models (decision trees, rules, linear models) and *post-hoc* explanations applied to opaque models (LIME, SHAP, saliency maps). Rudin [10] argues that, for high-stakes decisions, intrinsically interpretable models should be preferred to approximate post-hoc explanations — a thesis directly aligned with our architectural choice.

Our system belongs entirely to the first family: every detection is traceable from the initial RDF triple to the narrative report produced by the LLM. The SPARQL queries of the L1 workers are auditable, the triggering conditions of the L2 diagnosers are declarative, and the LLM only rephrases what the deterministic layers have already established. This property, absent from graph-embedding approaches where the decision is a black box, is intrinsic to the chosen architecture. The explainability score we define in Section 5.4 operationalizes this intuition: it counts the RDF triples actually traversed to produce each diagnosis, providing an objective, automatically computable measure that grows with the richness of the graph and requires no manual annotation.

1.3 Problem statement

The problem addressed in this project can be stated as follows: *how can we design a multi-agent system able to detect and diagnose anomalies in a knowledge graph representing a telecommunication network, while preserving strong explainability and keeping the complexity of distributed reasoning under control?*

This formulation reveals a central tension. On the one hand, diagnostic coverage requires increasingly specialized detection mechanisms: the more specialized the agents, the more they can capture fine, localized patterns. On the other hand, multiplying these agents increases the difficulty of orchestration, the risk of redundancy, and the complexity of the global interpretation. The problem is therefore not simply to *detect* anomalies, but to organize a set of symbolic sensors so as to produce a coherent, traceable and actionable diagnosis.

In our setting, this difficulty is twofold. We must first reason about *a priori* weaknesses, visible without any active incident, such as a lack of redundancy or missing information in the digital twin. We must then reason *a posteriori* about anomalies that have actually occurred, based on events, tickets or impacted services. The target architecture must therefore be able to combine several ontological facets, several analysis modes and several levels of reasoning.

1.4 Project objectives

The first objective of the project is to propose a functional multi-agent architecture for anomaly detection on a knowledge graph, using the NORIA-O ontology as the organizing principle of the detector catalogue. The second objective is to guarantee the explainability of the diagnostic pipeline, by maintaining a traceable symbolic decision chain from the level 1 SPARQL queries to the correlated level 2 diagnoses. The third objective is to measure the relevance of this architecture experimentally, using a realistic reference dataset, a controlled synthetic catalogue and a set of suitable KPIs.

In addition to these three main objectives, there is a methodological one: determining the right place for an LLM in such an architecture. Rather than making it the decision-making core of the system, we sought to assess under which conditions it could become useful as a synthesis layer, without compromising the auditability of the decisions produced upstream.

1.5 Main achievements

The project led to an advanced prototype, MASynReas, organized in three levels. Level 1 gathers specialized detection agents, distributed across the four facets of NORIA-O and the two analysis modes, *a priori* and *a posteriori*. Level 2 implements correlative diagnosers, responsible for aggregating level 1 signals around common anchors and assigning reliability, severity and priority scores. Level 3 is dedicated to narrative synthesis by an LLM, in an assistance role rather than a decision role.

On the experimental side, the project also produced a catalogue of synthetic datasets, KPI instrumentation, and a series of experiments comparing the MAS with different configurations involving an LLM. These achievements give the project a scope beyond a simple software prototype: they also make it a replicable evaluation basis. This reproducibility is reinforced by the availability of the source code in the project's GitHub repository, https://github.com/Blackbull11/MASynReas, which served as the common basis for development, for structuring the agent families, for generating the datasets and for instrumenting the experiments. In a work aimed precisely at explainability and auditability, the existence of such a repository is not secondary: it directly contributes to the scientific traceability of the project.

1.6 Report outline

This report is organized as follows. Section 2 reviews the main lessons from the intermediate phase of the project and the shifts that gradually led to the final architecture. Section 3 presents the overall architecture of MASynReas: its design principles, the three-level pipeline, the JaCaMo platform and the `PythonExecArtifact` artifact that serves as the interface between agents and the knowledge graph. Sections 4, 5 and 6 detail each level: respectively the 54 detection agents by NORIA-O facet, the 12 conditional-reasoning diagnosers based on the anchoring mechanism, and the LLM narrative synthesis module. Section 7 gathers the experiments carried out on `noria-0.2` and on the synthetic catalogue, in order to evaluate the complementarity of the families, the quality of the diagnoses, the robustness of the system and its comparison with different LLM-based baselines. Finally, Section 8 concludes by summarizing the contributions, the experimental results and the perspectives for future work.

2 Intermediate results and key lessons

This section reviews the main lessons learned during the intermediate phase of the project. Between the intentions stated at the launch of the PSC and the architecture actually built, several important shifts took place. Far from being mere contingent adjustments, they gradually clarified the nature of the problem to solve, the respective roles of the symbolic agents and of the LLM, and the conditions required for a credible experimental evaluation.

2.1 Strategic pivot: from the objective function to a working system

At the beginning of the project, a significant part of our thinking focused on defining a global *objective function* for the multi-agent system. The initial idea was to characterize the collective utility of the MAS by combining diagnostic quality, urgency management and computational cost, and then to derive an optimal organization of the agents from this abstract expression.

In practice, this direction quickly proved methodologically fragile. Defining an objective function before having a sufficiently mature implementation meant risking the optimization of an elegant formalism still disconnected from what could actually be implemented and measured. The project therefore made a *bottom-up* pivot. The priority

was no longer to solve the organizational problem in full generality from the outset, but to first build a working diagnostic system, then observe its empirical properties before returning to more abstract considerations.

This pivot proved scientifically fruitful. It replaced a purely conceptual ambition with a much more solid experimental basis: an explicit catalogue of detectors and diagnosers, instrumentation of execution times, a KPI framework, and a catalogue of controlled datasets. From this perspective, the three scores introduced at level 2 — reliability, severity, priority — can be read as a first pragmatic approximation of the initial ambition related to the objective function.

2.2 Learning JaCaMo through implementation

The implementation phase was also a gradual familiarization with the multi-agent stack itself. JaCaMo was not treated as a mere theoretical framework, but as the project's actual execution environment. Jason provided the BDI model used to express plans, activation conditions and the behavior of the controllers. CArtAgO supported the shared environment in which agents interact through artifacts.

In the final system, the central artifact is `PythonExecArtifact`. Its role is to provide the interface between the JaCaMo agents and the Python scripts responsible for executing SPARQL queries and producing JSON files. This Java–Python bridge was first introduced for pragmatic reasons, but it gradually became a structural asset of the project. It allows JaCaMo to be responsible for orchestration and coordination, while lightweight Python scripts handle graph access, result serialization and a large part of the detection logic. This separation greatly reduced the cost of extending the catalogue of level 1 agents.

2.3 From isolated cases to ontology-aligned families

Exploring the `noria-0.2` graph and the first prototypes showed that anomaly diagnosis could not be organized as a monolithic catalogue. The ontology itself suggested a more relevant structure. A missing interface, a burst of events, a hidden service dependency or a ticket without an associated procedure do not refer to the same semantic dimension of the network.

The organization of NORIA-O into four facets — structural, dynamic, functional and procedural — thus became a central design principle. This alignment had three beneficial effects. First, it made the detector catalogue conceptually clearer. Second, it improved the maintainability of the code and documentation. Finally, it made it possible to interpret the experimental results in a semantically structured way, rather than as a simple list of triggered agents.

2.4 Importance of the a priori / a posteriori distinction

A second major conceptual clarification emerged after structuring by ontological families: the distinction between *a priori* and *a posteriori* reasoning. The *a priori* mode concerns latent weaknesses visible in the graph independently of any ongoing incident. It thus targets intrinsic vulnerabilities of the network or missing information in the digital twin: missing interfaces, low redundancy, lack of functional coverage, procedural gaps or lack of observability.

The *a posteriori* mode, conversely, is centered on an anomaly that has already occurred. It starts from events, tickets, service disruptions or system changes to explain the cause or the operational consequences of the observed incident. This distinction proved important both architecturally and experimentally. It allows the same ontological family to support two very different analytical uses, and it explains part of the differences later observed in the benchmarks: *a priori* diagnosers often reason by absence, while *a posteriori* diagnosers more often rely on positive evidence present in the graph.

2.5 Designing datasets as a scientific instrument

Another essential lesson of the intermediate phase was that evaluation could not rely solely on the `noria-0.2` reference dataset. Although this graph is rich and realistic, it remains a single case, with a particular distribution of anomaly patterns. A project of this nature also requires controlled scenarios in which the expected diagnoses are explicitly known.

This realization led to building a full dataset framework. Two healthy reference graphs, *Base A* and *Base B*, were first defined as reusable starting points. From them, a catalogue of 27 scenarios was then generated to cover control cases, embedded diagnoses, robustness perturbations, ranking situations and scalability points. This catalogue became an experimental instrument in its own right, essential for the KPI instrumentation and for the systematic comparisons carried out in Section 7.

2.6 Lessons retained for the final architecture

The intermediate phase thus yielded several lessons that directly shaped the final architecture of MASynReas.

1. NORIA-O is not only a data model; it is also a relevant decomposition of the diagnostic space.
2. Deterministic scanning with SPARQL queries is particularly effective for anomalies that can be expressed as local configurations of the graph, or as missing relations or properties.
3. A useful diagnostic architecture needs at least two distinct symbolic levels: one level to detect, one level to aggregate the evidence and produce correlated hypotheses.
4. LLMs become useful when they receive a structured symbolic context, but they remain weak as exhaustive scanners of the graph.
5. A credible evaluation requires both a realistic reference graph and a catalogue of controlled synthetic scenarios.

These lessons serve as a transition to the following sections. Section 3 describes the resulting three-level architecture, then Sections 4, 5 and 6 detail each layer of the pipeline.

3 Architecture of the Multi-Agent System

3.1 Design principles

The architecture of MASynReas rests on four guiding principles, each associated with a measurable property of the system.

Separation of concerns. Each component of the pipeline has a clearly delimited function. At level 1, a worker corresponds to one specific detection pattern; at level 2, a diagnoser corresponds to a given correlation family; at level 3, the LLM's only function is narrative rephrasing. This separation strongly reduces coupling between modules and guarantees that a local change — adding a detector, adjusting a diagnoser, or modifying the synthesis prompt — introduces no side effects in the rest of the system. It also strengthens the introspectability of the MAS: for any diagnosis produced, one can trace back to the level 1 detectors involved, then to the specific SPARQL patterns that generated the corresponding evidence.

Detection parallelism. Level 1 agents do not communicate with each other during execution. Each queries Virtuoso independently and writes its result to a dedicated JSON file. JaCaMo's CArtAgO workspace allows them to coexist without blocking each other. The direct consequence is that the total detection time is bounded by the slowest agent, regardless of their number. On noria-0.2, this property yields speedups of 12.6× (a priori) and 18.7× (a posteriori) compared with an equivalent sequential execution (cf. Section 7).

Asynchronous BDI reactivity of the coordinator. The level 2 coordinator is a BDI agent (*Belief-Desire-Intention* paradigm implemented by Jason) that only activates once the workers have finished. It does not query Virtuoso directly: it reads the JSON files produced by the workers, evaluates cross-family logical conditions, and triggers the level 2 diagnosers accordingly. This asynchronous architecture completely decouples the coordinator from the order in which the workers finish.

Deterministic diagnosis before natural-language generation. The system first establishes an explicit diagnosis through symbolic reasoning, and only then rephrases it in natural language. In other words, the decision precedes the communication. This architectural constraint is essential to preserve a high level of explainability: the LLM does not infer the content of the diagnosis itself, it synthesizes results already established by levels 1 and 2. This prevents a plausible but insufficiently justified text from being mistaken for an auditable diagnostic decision.

3.2 Overview: a three-level pipeline

The hierarchy of the MAS serves three distinct reasoning functions. Level 1 *observes the graph exhaustively* from specialized angles; level 2 *interprets and correlates* the evidence produced by level 1; level 3 *communicates* the final diagnosis in a readable narrative form. This hierarchy is therefore not merely a technical stack: it corresponds to a decomposition of the reasoning itself, from local detection to reporting to the operator.

The pipeline runs sequentially across levels, but in parallel within level 1.

Level 1 — specialized detection. The first level gathers the 54 workers distributed across the four NORIA-O families (structural, dynamic, functional, procedural) and the two execution modes (a priori and a posteriori). Each worker encapsulates a SPARQL query or a small Python routine associated with a single anomaly pattern. Its output is a self-contained JSON file, which makes this level both massively parallelizable and very easy to audit.

Level 2 — diagnostic correlation. The second level does not start again from the raw graph, but from the consolidated level 1 results. It conditionally activates the relevant diagnosers, aggregates the evidence around common anchors (resource, application, service, zone, process), and produces higher-level diagnoses accompanied by reliability, severity and priority scores. Level 2 is thus where the synergy of the system takes its operational form: several local signals become a single, interpretable diagnosis.

Level 3 — narrative synthesis. The third level receives the already structured outputs of levels 1 and 2 and rephrases them in natural language. Its goal is not to discover new facts in the graph, but to present the results of the symbolic reasoning in a compact and intelligible way. It is therefore a communication layer, not a decision layer.

Concretely, execution proceeds as follows:

1. The configuration file `multiagentSystem.jcm` instantiates the agents relevant to the chosen mode and the workspace's shared artifact.
2. The artifact initializes the execution environment, clears the `results/` folder to guarantee a clean run, then publishes the active mode (`apriori` or `aposteriori`).
3. The level 1 workers corresponding to this mode start, launch their Python scripts, query the Virtuoso endpoint and write their JSON results in parallel.
4. Once this first computation front is complete, the level 2 controller reads the available results and triggers only the diagnosers whose activation conditions are satisfied.

5. The active diagnosers write their outputs to `results/level2/`, with the diagnostic anchors and the associated scores.

6. Finally, level 3 aggregates the L1 and L2 outputs as prompt input and, when invoked, produces a narrative report intended for human reading.

This pipeline runs in *a priori* mode (preventive analysis of the permanent structure of the graph, without incident context) or in *a posteriori* mode (analysis contextualized by tickets, incidents or change signals present in the graph).

3.3 The JaCaMo platform

JaCaMo [5] integrates three complementary components.

Jason provides the agent execution engine following the BDI paradigm: each agent has beliefs, desires (*goals*) and intentions (*plans*). In MASynReas, level 1 workers have a single plan: run their Python script at initialization. The level 2 coordinator has richer logic: it waits for the workers' completion signals, loads the JSON results and selects the diagnosers to activate.

CArtAgO (*Common ARTifact infrastructure for AGents Open systems*) manages shared environments in the form of *artifacts*. The central artifact of MASynReas is `PythonExecArtifact`, which each worker observes (`focus`) and through which it invokes its Python script. This artifact is the bridge between the agent logic (AgentSpeak) and the Python ecosystem (HTTP requests, numerical computation, JSON writing): JaCaMo agents never directly handle sockets or system files.

Moise provides the organization primitives (roles, missions, norms). In the current version, the organization is conceptually structured into three roles (L1 Detector, L2 Diagnostician, L3 Narrator) but without a formal Moise specification. This formalization is one of the main perspectives of the project (cf. Section 8).

3.4 The agent–environment interface: PythonExecArtifact

The `PythonExecArtifact` embodies the environment for the agents of the MAS. This environment is an RDF knowledge graph structured by the NORIA-O ontology and exposed, in our experimental setup, by a local Virtuoso SPARQL endpoint (`http://localhost:8890/sparql`). Each run consists of loading a given graph — either the `noria-0.2` reference dataset or one of the synthetic datasets of the catalogue — then letting the agents query this same state of the world through the shared artifact. Agents therefore never directly handle Turtle files or HTTP connections: all interaction with the graph goes through this interface layer.

The `PythonExecArtifact` exposes three main operations.

`execPython(scriptPath)` is the central operation: it creates a child process via `ProcessBuilder`, runs the designated Python script and forwards its standard output to the workspace. This *Java–Python bridge* is the architectural key of the system: it lets agents use the entire Python ecosystem without these dependencies entering the Jason layer. Adding a new detector amounts to writing a Python script and declaring an agent in the JCM, without any Java recompilation.

`execQuery(sparqlQuery)` submits a SPARQL query directly via Jena ARQ without going through Python. It is used for lightweight queries that do not require Python processing.

`writeReport/appendReport` write to the run's consolidated report file, allowing agents to leave a narrative trace without going through the LLM.

In the implementation adopted for MASynReas, this interface also plays a role of experimental normalization: it requires all agents to read the same graph, publish their outputs in a common results tree, and run in a homogeneous setting regardless of their NORIA-O family. It is therefore both a technical bridge and a guarantee of consistency between the different levels of the pipeline.

All level 1 agents follow the same pattern in the JCM:

Listing 1: Excerpt from `multiagentSystem.jcm`: each agent observes the same artifact `w`. Its identity in the JCM is enough to resolve the path of its Python script by convention.

```
mas multiagentSystem {

    agent orphan_resource_detector   { focus: w.pyexec }
    agent missing_interface_detector { focus: w.pyexec }
    // ... (54 agents, same structure)

    workspace w {
        artifact pyexec : PythonExecArtifact("http://localhost:8890/sparql")
    }
}
```

This *convention over configuration* approach eliminates all boilerplate: adding a 55th agent only requires one line in the JCM.

4 Level 1: Detection Agents by NORIA-O Facet

4.1 General organization: a priori and a posteriori

The 54 level 1 agents are organized along two orthogonal dimensions.

The first is the **family**, aligned directly with the four facets of NORIA-O: structural, dynamic, functional, procedural. Each family monitors the subset of the ontology's classes and properties that corresponds to it. This dimension is always explored in full: when a family is activated, all the agents it contains for the mode considered are instantiated and run.

The second is the **analysis mode**:

- **A priori**: the agent queries the permanent structure of the graph without incident context. Its SPARQL query contains no reference to an active `TroubleTicket` or `EventRecord`. The goal is the proactive detection of vulnerabilities, gaps and inconsistencies *before* an incident occurs. In practice, this mode most often corresponds either to the detection of weaknesses inherent to the network, or to revealing missing information in its graph representation, that is, in its digital twin.
- **A posteriori**: the agent starts from an active dynamic entity (`TroubleTicket` or `EventRecord`) and traverses the graph to explain the observed symptom by linking it to structural, functional or procedural causes.

Unlike the *family* dimension, the *analysis mode* dimension is selected explicitly by the user in the MAS settings before execution.

Family	A priori	A posteriori	Total
Structural	10	7	17
Dynamic	3	12	15
Functional	7	9	16
Procedural	3	3	6
Total	**23**	**31**	**54**

Table 1: Distribution of the 54 level 1 agents by family and mode.

4.2 Structural family

The structural family monitors the physical topology of the network: resources (`noria:Resource`), interfaces (`noria:NetworkInterface`), links (`noria:NetworkLink`) and their support relations (`seas:subSystemOf`, `noria:networkInterfaceOf`). In NORIA-O, this aspect models the materiality of the ICT system: the technical objects, their connectivity, their hierarchy and their physical location.

A priori agents. The a priori agents of this family mainly look for *intrinsic topological weaknesses* and *modeling blind spots* in the digital twin.

Detected pattern	Agent
Structurally isolated resource, with no interface, management unit, parent, child or application support	`orphan_resource_detector`
Resource without an explicit interface	`missing_interface_detector`
Interface not attached to a resource	`orphan_interface_detector`
Attached interface but without a network link	`unconnected_interface_detector`
Network link with fewer than two endpoints	`incomplete_network_link_detector`
Apparently subordinate resource without an explicit parent	`missing_parent_resource_detector`
Application without visible supporting resource	`application_without_support_detector`
Resource without a management unit	`unmanaged_resource_detector`
Critical application with insufficiently redundant support	`missing_redundancy_detector`
Critical application combining business criticality and structural weakness	`criticality_structural_weakness_detector`

Table 2: A priori structural catalogue.

A posteriori agents. The a posteriori agents contextualize structural weaknesses around resources or applications already involved in an incident.

Detected pattern	Agent
Incident-affected resource with very weak connectivity	`isolated_incident_resource_detector`
Incident on a resource connected to an incomplete link	`incident_on_incomplete_link_detector`
Incident-affected resource with high downstream impact on resources or applications	`high_impact_resource_detector`
Incident-affected resource without visible redundancy	`no_redundancy_incident_detector`
Incident involving an application with an inconsistent support mapping	`application_mapping_inconsistency_detector`
Spatial concentration of incidents in the same area	`spatial_incident_cluster_detector`
Incident escalation from a child component to a structuring parent resource	`child_component_incident_escalation_detector`

Table 3: A posteriori structural catalogue.

Representative agent: `orphan_resource_detector`. This agent detects resources that belong to no operational structure. The query combines five `FILTER NOT EXISTS` clauses:

Listing 2: Query of the orphan resource detector. This pattern of reasoning by absence is characteristic of a priori agents.

```
SELECT ?resource WHERE {
  ?resource a noria:Resource .
  FILTER NOT EXISTS {
    ?interface noria:networkInterfaceOf ?resource . }
  FILTER NOT EXISTS { ?resource noria:resourceForApplication ?app . }
```

```
  FILTER NOT EXISTS { ?resource noria:resourceManagedBy ?orgUnit . }
  FILTER NOT EXISTS { ?resource seas:subSystemOf ?parent . }
  FILTER NOT EXISTS { ?child   seas:subSystemOf ?resource . }
}
```

This pattern of *reasoning by absence* — detecting what is missing rather than what is present — is characteristic of a priori agents. On noria-0.2, it identifies 14 orphan resources.

4.3 Dynamic family

The dynamic family monitors temporal entities: events (`noria:EventRecord`) with their timestamp (`noria:loggingTime`) and severity (`noria:alarmPerceivedSeverity`), and planned changes (`noria:ChangeRequest`). In NORIA-O, this aspect models how the system evolves over time: occurrence of events, changes, escalations, recurrences and possible causal successions between dynamic facts.

A priori agents. The dynamic a priori agents mainly detect observability or temporal completeness defects in the graph.

Detected pattern	Agent
Change request without a complete effective time	`change_without_effective_time_detector`
Event without a timestamp	`event_without_timestamp_detector`
Event without a related network element	`event_without_related_element_detector`

Table 4: A priori dynamic catalogue.

A posteriori agents. The dynamic a posteriori agents form the richest temporal detection subsystem of level 1. They can be grouped around three main aspects.

- **Change–incident correlation**: the incident occurs shortly after a change, several changes overlap, or a silent degradation follows a modification.
- **Volume and simultaneity patterns**: event bursts, closely spaced repetitions, or incidents affecting several elements in the same time window.
- **Incident evolution and propagation**: state flapping, reopened ticket, stale ticket, parent–child escalation, missing escalation of a critical event, or propagation to adjacent elements.

Detected pattern	Agent
Incident occurring shortly after a change on the same element	`change_followed_by_incident_detector`
Overlapping changes on the same element	`change_overlap_conflict`
Critical event followed by a ticket escalation deemed significant	`critical_event_ticket_escalation_detector`
Burst of events over a short window	`event_burst_detector`
Rapid oscillation between contradictory states	`flapping_state_detector`
Short propagation to an adjacent element	`incident_propagation_detector`
Synchronous incidents on several distinct elements	`multi_element_synchronous_incident_detector`
Parent–child temporal escalation	`parent_child_event_escalation_detector`
Reopened incident on the same element	`reopened_incident_detector`
Closely spaced repetition of similar events	`repeated_similar_event_detector`
Silent degradation after a change	`silent_degradation_after_change_detector`
Persistent incident with no visible resolution	`stale_incident_detector`

Table 5: A posteriori dynamic catalogue.

4.4 Functional family

The functional family monitors application dependencies: services (`noria:Service`), modules (`noria:ApplicationModule`), applications (`noria:Application`) and their business relations (`noria:resourceForApplication`, `noria:applicationModule`). In NORIA-O, this aspect models the business-meaning layer of the network: support chains between technical resources, applications, modules and the services exposed to users.

A priori agents. The functional a priori agents detect modeling gaps and coverage weaknesses in the service–module–application–resource chain.

Detected pattern	Agent
Application without a functional module	`application_without_module_detector`
Application without visible supporting resource	`application_without_resource_detector`
Resource mapped to several applications	`duplicated_functional_mapping_detector`
Module attached to several parent services	`inconsistent_service_hierarchy_detector`
Service supported by too few resources	`over_concentrated_service_detector`
Service without an attached application module	`service_without_application_detector`
Service without resource coverage along the whole functional chain	`service_without_resource_coverage_detector`

Table 6: A priori functional catalogue.

A posteriori agents. The functional a posteriori agents reason about the business impact of incidents and their propagation through application dependencies.

Detected pattern	Agent
Application incident without visible supporting resource	`application_incident_without_resource_detector`
Potential propagation of a failure to several services	`cascading_service_failure_detector`
Contradictory application states at the same time	`conflicting_application_state_detector`
Hidden dependency between services	`hidden_service_dependency_detector`
Convergence of incidents at module level	`module_level_incident_aggregation_detector`
Resource event without traceable service impact	`resource_event_without_service_impact_detector`
Service impacted by several distinct resources	`service_impacted_by_multiple_resources_detecto`
Impacted service without associated ticket escalation	`service_without_ticket_escalation_detector`
Recurrently affected service	`service_with_repeated_incident_detector`

Table 7: A posteriori functional catalogue.

4.5 Procedural family

The procedural family monitors operational processes: remediation procedures (`noria:OperationPlan`), change requests (`noria:ChangeRequest`) and their links with tickets and resources. In NORIA-O, this aspect models how the organization reacts to anomalies: preparation of procedures, traceability between events and tickets, and the connection between incidents and planned operations.

A priori agents. The procedural a priori agents detect signs of incomplete operational readiness even before an incident is analyzed.

Detected pattern	Agent
Change request without a scheduled time slot	`change_request_without_scheduled_time_detector`
Procedure without visible link to a resource type	`procedure_not_linked_to_resource_type_detector`
Ticket without a procedure assignable through its linked events	`ticket_without_assigned_procedure_detector`

Table 8: A priori procedural catalogue.

A posteriori agents. The procedural a posteriori agents detect traceability breakdowns or anomalies in the active operational chain.

Detected pattern	Agent
Change linked to several simultaneous incidents	`change_linked_to_multiple_incidents_detector`
Event-type incident without an associated ticket	`incident_without_ticket_detector`
Ticket without a linked event	`ticket_without_linked_event_detector`

Table 9: A posteriori procedural catalogue.

5 Level 2: Conditional Reasoning Agents

5.1 Motivation and correlation principle

Level 2 agents form the *deterministic diagnosis* layer of the system. While level 1 extracts elementary patterns, level 2 aims to turn this local evidence into interpretable hypotheses that can be ranked. Its philosophy is therefore the following: a relevant diagnosis is not the mere repetition of a level 1 alert, but the coherent aggregation of several signals, possibly from different families, around the same operational situation.

This correlation relies on two central notions. The first is the *anchor*: the common entity or scope on which several level 1 results converge. Depending on the case, this anchor can be a resource, an application, a service, a ticket, a change request, a network link, a location or a procedural scope. The second is the *Single Point of Failure* (SPOF): a situation in which a single weak component concentrates enough evidence to make it a plausible local cause of failure.

A simple example illustrates this role. `isolated_incident_resource_detector` reports that an incident-affected resource appears weakly connected in the topology. `no_redundancy_incident_detector` reports that this same resource has no visible backup path. Taken separately, these results remain local warnings. Combined on the *same* anchor of type `Resource`, they support a stronger diagnosis: the resource is most likely a **Single Point of Failure (SPOF)**. It is precisely this qualitative leap — moving from a set of atomic results to a synthetic interpretation — that justifies the existence of level 2.

5.2 Common framework `common.py` and execution principle

Level 2 orchestration starts with the `level2_controller.asl` agent. This controller waits until all level 1 detectors of the selected mode have finished, then triggers only the catalogue of diagnosers of the same mode. In *a priori* mode, it thus launches five diagnosers; in *a posteriori* mode, it launches seven. In the first case, it automatically stops the MAS once these diagnosers have finished.

On the Python side, all level 2 diagnosers are built on a common framework: `src/agt/level2/common.py`. This file factors out the deterministic logic shared by the whole catalogue. Each diagnoser declares its inputs, its activation rules, its possible anchors, how to compute its diagnosis, and the metadata it exports.

The framework first relies on an `InputSpec` structure, which associates with each level 1 agent used:

- the JSON file to read;
- the NORIA-O family of origin;
- the analysis mode;
- the evidence role assigned to this agent;
- the anchor extraction rules;
- optional *tags* used to enrich the severity assessment.

Each level 1 agent involved in a diagnosis is assigned one of three roles: **P** for *Primary evidence*, **S** for *Supporting evidence*, and **C** for *Contextual evidence*. `P` evidence carries the main direct proof, `S` evidence reinforces the diagnostic hypothesis, and `C` evidence adds impact or prioritization context. These roles are used both to decide whether the diagnoser should trigger and to weight the scores produced afterwards.

The common execution principle is as follows:

1. the level 1 JSON files referenced by the diagnoser are loaded;
2. the records they contain are normalized;
3. potential anchors are extracted according to declarative rules;
4. results are aggregated by anchor;
5. an activation rule decides whether the diagnoser should produce at least one output;
6. a candidacy rule decides, anchor by anchor, whether an explicit diagnosis should be created;
7. for each retained diagnosis, the reliability, severity and priority scores are computed, then exported to `results/level2/[mode]/[diagnoser].json`.

Pipeline example: `single_point_of_failure_diagnoser`**.** The `single_point_of_failure_diagnoser` is a good example, because its behavior is simple and highly interpretable. It uses five level 1 results, all from the a posteriori structural family: isolated incident-affected resource, incident on an incomplete link, high-downstream-impact resource, lack of visible redundancy, and escalation from a child component. In the current implementation, four of these inputs are primary evidence and one is supporting evidence. The diagnoser activates as soon as an anchor concentrates at least two pieces of primary evidence, then exports a diagnosis for each anchor satisfying the same criterion.

Table 10 gives the excerpt of the mapping matrix associated with this diagnoser.

Level 1 agent	**Role for** `single_point_of_failure`
`isolated_incident_resource_detector`	P
`incident_on_incomplete_link_detector`	P
`high_impact_resource_detector`	P
`no_redundancy_incident_detector`	P
`child_component_incident_escalation_detector`	S

Table 10: Excerpt of the mapping matrix for `single_point_of_failure_diagnoser`.

This example clearly shows the role of the common framework: it does not impose a particular diagnosis, but an *execution grammar* common to the whole catalogue. Each diagnoser then specializes this grammar through its `InputSpec`, its activation rules and its semantic domain.

5.3 Catalogue of implemented diagnosers

Level 2 currently contains twelve diagnosers: five in *a priori* mode and seven in *a posteriori* mode. They are deliberately **cross-cutting with respect to the NORIA-O families**. In other words, they are not organized by structural, dynamic, functional or procedural facet, but by *diagnostic family*. This choice is essential: a diagnosis is only relevant if it aggregates several elementary patterns which, taken in isolation, are not always enough to produce a satisfactory explanation.

A priori diagnosers. The a priori diagnosers seek to establish latent weaknesses of the system before any explicit incident.

Diagnoser	Diagnosis produced	Triggering condition (summary)
structural_fragility_diagnoser	Diagnoses a structural fragility on a resource or support chain accumulating several topological weaknesses.	Triggers when the same anchor concentrates several primary or supporting structural signals, such as isolation, missing interface, weak parenting or missing redundancy.
critical_service_exposure_diagnoser	Diagnoses a critical exposure of a business service or application whose technical support is too weak.	Activates when business-criticality evidence converges with coverage or redundancy gaps on the same application or service.
observability_gap_diagnoser	Diagnoses an observability deficit that will compromise the later reconstruction of an incident.	Activates when events or changes simultaneously lack a timestamp, a related element or usable temporal information.
procedural_unreadiness_diagnoser	Diagnoses operational unreadiness due to incomplete procedures or scheduling.	Activates when tickets, changes or procedures jointly show readiness gaps, such as the lack of an assignable procedure or of a scheduled time slot.
functional_mapping_gap_diagnoser	Diagnoses incomplete or inconsistent functional traceability between services, modules, applications and resources.	Activates when mapping breaks or ambiguities converge on the same functional chain.

Table 11: Catalogue of a priori level 2 diagnosers.

A posteriori diagnosers. The a posteriori diagnosers interpret incidents, degradations or operational failures already present in the graph.

Diagnoser	Diagnosis produced	Triggering condition (summary)
`single_point_of_failure_diagnoser`	Diagnoses a local situation in which a weak component is a single point of failure.	Activates when the same resource concentrates at least two pieces of primary evidence of incident-related fragility, such as isolation, lack of redundancy or an incomplete link.
`change_induced_incident_diagnoser`	Diagnoses a change as a plausible cause or amplifier of an incident.	Activates when a change, overlapping changes or a post-change degradation converge on the same change request or the same linked element.
`service_cascade_diagnoser`	Diagnoses an upward propagation of an incident through a shared service support chain.	Activates when several functional or dynamic propagation signals cluster around a common service, module or supporting resource.
`traceability_breakdown_diagnoser`	Diagnoses a systemic breakdown in the incident–event–ticket–procedure traceability chain.	Activates when unticketed incidents, tickets without a linked event, or service impacts without procedural escalation combine within the same operational scope.
`unstable_component_diagnoser`	Diagnoses chronic instability rather than a one-off failure.	Activates when evidence such as bursts, repetitions, flapping, reopening or abnormal age converges on the same component.
`application_support_failure_diagnoser`	Diagnoses an application failure aggravated by a missing, ambiguous or contradictory support chain.	Activates when application incidents converge with mapping or support-impact anomalies on the same application.
`local_infrastructure_cluster_diagnoser`	Diagnoses a disrupted local infrastructure area rather than independent failures.	Activates when spatial concentration, local propagation, structural escalation or incomplete topology converge on the same location or resource area.

Table 12: Catalogue of a posteriori level 2 diagnosers.

5.4 Approximating the objective function: scoring the diagnoses

A level 2 diagnosis is not only meant to exist or not; it must also be qualified. For this reason, each level 2 output comes with three distinct scores: a reliability score, a severity score, and a priority score. This three-way split meets a clear conceptual need. A diagnosis can be very well supported without being particularly serious; conversely, it can be potentially critical while still resting on limited evidence. Priority then combines these two dimensions for operational triage.

Reliability score. The reliability score answers the question: *how strongly do the level 1 results support this diagnostic hypothesis?* In the current implementation of `common.py`, it is computed from the quantity and type of evidence present around the same anchor. Each piece of primary evidence (`P`) adds +20, each piece of supporting evidence (`S`) adds +10. A cross-family bonus is then applied: +10 if at least two NORIA-O families are represented, +20 if at least three are. Finally, an anchoring bonus is added: +15 for anchors considered strong (`Resource`, `Application`, `Service`, `ChangeRequest`, `TroubleTicket`, `Location`) and +5 for weaker or more diffuse anchors. This gives:

$$\texttt{reliability_score} = \min(100,\ 20 \times \#P + 10 \times \#S + \texttt{family_bonus} + \texttt{anchor_bonus})$$

The reliability score therefore measures the *evidential strength* of the diagnosis, not its operational seriousness.

Severity score. The severity score answers a different question: *if this diagnosis is true, how serious is the situation?* In `common.py`, this score is obtained by aggregating several components:

- a criticality component, derived from fields such as `criticality`, `businessCriticality`, `severity` or `priority` present in the results;
- a *breadth* component, based on the total number of aggregated elementary results;
- a concentration component, enriched by detectors carrying the `concentration` tag;
- a temporal aggravation component, fed by detectors tagged `temporal`;
- a governance weakness component, fed by detectors tagged `governance`;
- a base bonus specific to the diagnostic family.

Schematically, we therefore have:

$$\texttt{severity_score} = \min(100, \texttt{criticality}+\texttt{breadth}+\texttt{concentration}+\texttt{temporal}+\texttt{governance}+\texttt{base_bonus})$$

This score expresses the intrinsic or systemic seriousness of the diagnosis, even if its level of evidence remains moderate.

Priority score. Finally, the priority score answers the question: *which diagnosis should be inspected first?* In the current implementation, it is computed as a linear combination of the two previous scores:

$$\texttt{priority_score} = \text{round}(0.55 \times \texttt{reliability_score} + 0.45 \times \texttt{severity_score})$$

This score is therefore neither a duplicate of reliability nor a mere synonym of severity. It serves to *rank* competing diagnoses and allows level 2 to produce a relevant inspection order in scenarios with multiple diagnoses.

As explained in Section 2.1, this priority score is a *pragmatic approximation of the objective function* initially envisaged for the multi-agent system. We do not yet have a fully formalized global utility function in the organizational sense; however, priority already synthesizes, in an operational way, two essential dimensions of this utility: confidence in the diagnosis and the potential cost of ignoring it.

6 Level 3: Narrative Synthesis by an LLM

6.1 Separating decision and communication

Level 3 is the *communication* layer of the system. It receives all the results of levels 1 and 2 and produces a natural-language report for network operations teams.

The central architectural property is that this level **makes no decision**: the diagnosis is entirely established by levels 1 and 2 before the LLM is invoked. This *decision / communication* separation preserves the full explainability of the pipeline: every statement in the narrative is traceable to a worker or diagnoser JSON result, itself traceable to a specific RDF triple of the graph. This property contrasts with approaches in which an LLM is consulted directly for the diagnosis; the subsection on replacing level 1 with an LLM detector (Section 7.4.3) illustrates the limits of that approach (recall = 0.21 at best, reasoning by absence structurally impossible).

6.2 Prompt construction and model

The `diagnosis_llm.py` script aggregates the JSON outputs of the active agents into a prompt structured in three blocks:

1. *Level-1 Alerts*: list of active L1 detections (anomaly type, entity, family);
2. *Level-2 Diagnoses*: triggered L2 diagnosers, anchor entities and reliability, severity and priority scores;
3. Task instruction: produce 3 to 5 sentences of prose, cite the exact resource identifiers, state the overall severity, and do not use bullet points.

The model used in the prototype's local configuration is **Llama 3.2:3b**, served locally via **Ollama**. This choice meets two operational constraints: confidentiality of network data (no data is sent outside) and CPU compatibility (3 billion parameters in Q4 quantization, inference in a few tens of seconds without a GPU). For the comparative LLM/MAS experiments of Section 7, **Mistral-7B-Instruct-v0.3** in float16 on GPU (`jaguar.polytechnique.fr`) is used to explore the capabilities of a more powerful model under optimal conditions.

6.3 Example output on the noria-0.2 scenario

Applied to the reference scenario (L2 failure on `RES_TOY_as2`, loss of access to `APP_TOY_PFS01`), Llama 3.2:3b produces:

Listing 3: Narrative report produced by Llama 3.2:3b on the noria-0.2 scenario. The resource identifiers are correctly reproduced.

```
Diagnostic Report – Network Incident Analysis

Based on the structural and dynamic evidence collected, the primary
incident affects resource RES_TOY_as2, which presents a critical
isolation pattern: no redundant network path is available and the
associated alarm (SYS_ERROR: L2 table overflow) has propagated
to RES_TOY_srv1.

Application APP_TOY_PFS01, which depends on RES_TOY_as2 for its
infrastructure, is directly impacted. Trouble ticket TT_TOY2022TT
documents this failure but no repair procedure has been assigned.

Recommended action: escalate to the support team responsible for
RES_TOY_as2; restore the L2 routing table and verify redundancy
configuration before returning the resource to service.
Overall severity: CRITICAL.
```

The narrative correctly reports the faulty resource (`RES_TOY_as2`), the impacted application (`APP_TOY_PFS01`), the trouble ticket (`TT_TOY2022TT`) and the lack of a remediation procedure. These four elements are directly traceable to the JSON outputs of the workers `no_redundancy_incident_detector`, `incident_propagation_detector`, `service_without_ticket_escalation_detector` and `ticket_without_assigned_procedure_detector`.

7 Experiments and Results

All experiments were carried out with a local Virtuoso endpoint (`localhost:8890`), with Python 3.13 and JaCaMo 1.2, combining the **noria-0.2** reference graph [**noria-dataset**] and the catalogue of synthetic datasets presented below.

7.1 Tested dataset catalogue and experimental processes

7.1.1 Two complementary experimental bases

The experimental setup relies on two complementary databases.

The first is the **NORIA-O 0.2** graph [7], used as a realistic reference case. It contains **3,152 RDF triples** and instantiates the four facets of the NORIA-O ontology [13]:

- **Structural** — physical resources, interfaces, network links, applications and support relations;
- **Dynamic** — timestamped events and planned changes;
- **Functional** — services, application modules and support dependencies;
- **Procedural** — trouble tickets and operational management elements.

This graph is mainly used to validate the architecture qualitatively on a credible use case, and to observe how the different layers of the MAS behave on data close to the business domain.

The second base is a **catalogue of 27 synthetic datasets**, built from two healthy reference graphs (Base A and Base B), then derived through controlled mutations. Each scenario is documented by a `dataset.ttl` graph, a `manifest.json`, expected level 1 and level 2 outputs, and a `README.md` file. These expected outputs form the *ground truth* of the test bench: they were defined manually when designing each scenario, by building the graph around a specific target anomaly and identifying, from the formal rules of the modules concerned, the diagnosers that should theoretically trigger. The catalogue thus provides a genuine test bench for the MAS.

7.1.2 Structure of the synthetic catalogue

The catalogue was organized into scenario families, each targeting a distinct aspect of testing the system.

Scenario family	Experimental objective	# datasets
Control and activation	Check that the MAS stays silent on healthy or weak-evidence graphs, and test the activation conditions	3
Embedded a priori diagnoses	Exercise the main diagnosers of latent system weaknesses	7
Embedded a posteriori diagnoses	Exercise the main diagnosers of incidents, propagation and operational breakdown	8
Robustness	Test the effect of missing data, noise and duplications	4
Ranking, calibration and scalability	Measure the ranking of diagnoses and the evolution of execution times	5

Table 13: Organization of the synthetic catalogue of 27 datasets.

This organization shows that the catalogue was not conceived as a simple collection of toy graphs, but as a test setup covering most of the properties we want to establish for MASynReas: diagnostic accuracy, conservative behavior, robustness, prioritization and temporal stability.

7.1.3 Common thread of the experiments

The experiments presented in this section answer four complementary questions:

1. **Does the NORIA-O 0.2 case validate the basic architectural choices?**
2. **Is the L1+L2 pipeline accurate, robust and stable on a controlled benchmark?**
3. **What added value does the MAS bring to a synthesis LLM?**
4. **Can an LLM replace the MAS as a detector or as a diagnoser?**

The rest of the section follows this progression directly.

7.2 First architectural validations on NORIA-O 0.2

This subsection answers question 1 of § 7.1.3.

Before using the synthetic catalogue, two validations on `noria-0.2` justify the most structuring architectural choices of the MAS: on the one hand, the decomposition of level 1 into independent agents, and on the other hand, its factorization into four NORIA-O families.

7.2.1 Multi-agent parallelism

To quantify the gain brought by JaCaMo's parallel architecture, we compare the multi-agent execution with a sequential baseline that runs the same Python scripts one after the other. Both configurations share exactly the same queries and the same detection code; only the scheduling strategy differs.

Mode	Agents	Sequential	Parallel (th.)	Speedup
A priori	23	4,866 ms	386 ms	**12.6**×
A posteriori	31	6,514 ms	349 ms	**18.7**×

Table 14: Sequential baseline vs theoretical parallelism on `noria-0.2`.

The multi-agent architecture provides a speedup of **12.6**× in a priori mode and **18.7**× in a posteriori mode. This property is enough to show that the choice of independent worker agents is not only a matter of modularity: it also provides a concrete performance advantage.

Takeaway. The parallel level 1 is justified both as a software structure and as a mechanism for scaling to a large number of detectors.

7.2.2 Complementarity of the four agent families

The second qualitative validation aims to establish that partitioning level 1 along the four facets of NORIA-O is not a mere filing choice, but a useful property of the system. The goal is therefore to check that the structural, dynamic, functional and procedural families do bring different viewpoints on the same graph, and that they actually prepare interesting correlations for level 2.

The protocol consists of running all detectors on `noria-0.2`, then aggregating the results by family to identify which facets produce signals, on which entities, and with which overlaps.

Family	Active agents	Detections	Total agents	Most active agent
Structural	2	17	10	`missing_interface` (14)
Dynamic	1	10	3	`event_without_related_element` (10)
Functional	2	7	7	`inconsistent_service_hierarchy` (4)
Procedural	1	2	3	`ticket_without_assigned_procedure` (2)
Total	**6**	**36**	**23**	

Table 15: Results of the level 1 agents in a priori mode on noria-0.2.

A priori mode. The a priori mode shows clear non-redundancy between families. Structural weaknesses, observability gaps, functional inconsistencies and procedural deficiencies are expressed neither on the same objects nor through the same patterns. Some applications, such as `APP_TOY_DC01`, `APP_TOY_DC02` or `APP_TOY_MON01`, are however seen by several families at once. These overlaps are natural targets for level 2, which can then turn several local alerts into a single diagnosis of fragility or exposure.

Family	Active agents	Detections	Total agents
Structural	**2**	**10**	7
Dynamic	0	0	12
Functional	**7**	**19**	9
Procedural	0	0	3

Table 16: Results of the level 1 agents in a posteriori mode on noria-0.2.

A posteriori mode. In a posteriori mode, the structural and functional families are active, while the dynamic and procedural families remain silent on this case study. This lack of triggering does not indicate a weakness of the architecture, but a property of the dataset: `noria-0.2` contains neither a sufficiently dense temporal history, nor `noria:Change` objects, nor explicit breaks in ticket–event links. In a richer production graph, these two families would also be triggered.

> *This validation confirms that dividing level 1 into four families is relevant. It guarantees both the specialization of the sensors and the possibility of cross-family correlations at level 2, which would be out of reach in a monolithic architecture.*

7.3 Various evaluations on the synthetic catalogue

This subsection answers question 2 of § 7.1.3: is the L1+L2 pipeline accurate, robust and stable on a controlled benchmark?

7.3.1 Selected metrics

The synthetic catalogue makes it possible to move from qualitative validation to systematic evaluation. The selected metrics correspond to different questions about the behavior of the MAS.

Let TP be the number of correctly triggered diagnosers, FP the number of wrongly triggered diagnosers, and FN the number of expected diagnosers that were not triggered. We define:

$$\text{Precision}_{L2} = \frac{TP}{TP + FP}, \quad \text{Recall}_{L2} = \frac{TP}{TP + FN}, \quad F1_{L2} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

For scenarios with multiple diagnoses, we also use the *MRR* (*Mean Reciprocal Rank*):

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q}$$

where rank_q is the rank, in the list sorted by priority score, of the first expected diagnoser for scenario q. Finally, robustness and execution time measurements complete the analysis, to evaluate respectively the stability of the system with degraded data and its compatibility with operational use.

7.3.2 Overall diagnostic accuracy

The main experiment consists of running the MAS on the 27 datasets of the catalogue, then comparing the diagnosers actually triggered with the ground truth defined for each scenario. The overall results are summarized in Table 17.

Mode	Precision_{L2}	Recall_{L2}	F1_{L2}
A posteriori	0.825	0.808	**0.815**
A priori	0.177	0.740	0.196
Overall	0.394	0.542	0.456

Table 17: Performance of the level 2 diagnosers on the synthetic catalogue. The "Overall" row is computed by micro-averaging (TP, FP, FN summed over all mode × dataset pairs) and does not correspond to an average of the per-mode F1 scores; it should be read together with the per-mode results.

The first striking result is the very good behavior of the a posteriori level 2, with a macro-F1 of 0.815. Diagnosers based on positive evidence present in the graph — single point of failure, change-induced incident, service cascade, traceability breakdown — trigger with high precision and a recall close to 0.81. Conversely, the aggregated a priori performance remains limited. This result does not mean that the correlation principle is ineffective in a priori mode; rather, it highlights a few activation conditions that are still too broad, notably for `procedural_unreadiness_diagnoser`, which accounts for a large share of the false positives.

Takeaway. Level 2 is already robust and relevant for scenarios contextualized by incidents or tickets. The remaining work concerns tightening some a priori diagnosers rather than the architecture itself.

7.3.3 Ranking of diagnoses

Several scenarios of the catalogue were designed to produce several competing diagnoses on the same graph. In this case, the experimental question is no longer only to find the right diagnosers, but to rank them correctly. Table 18 summarizes this evaluation.

Subset	Top-1 Accuracy	MRR
Multi-diagnosis scenarios	1.000	0.833

Table 18: Ranking quality on scenarios with multiple diagnoses.

The system systematically finds the main diagnosis in first position on the evaluated cases, and on average places the expected diagnoses at high ranks. This point is important: from an operational perspective, the value of level 2 is not to produce a simple list of plausible patterns, but to quickly bring out the dominant hypotheses.

Takeaway. The priority score is not just a presentation artifact; it effectively plays its role as a simplified triage function.

7.3.4 Robustness to missing data and noise

Four specific scenarios were built to test the behavior of the system when the graph is degraded: removal of structural or procedural links, addition of off-target events, duplication of records.

Dataset	Perturbation	$F1_{L2}$
DS19	Missing structural and temporal data	**1.00**
DS20	Incomplete procedural links	**1.00**
DS21	Irrelevant events (noise)	**1.00**
DS22	Duplicated records (noise)	**1.00**

Table 19: Robustness of the MAS on degraded-data scenarios.

In all four cases, the observed behavior is exactly the expected one. Noisy scenarios do not cause spurious triggers, while scenarios with incomplete information do not lead to unwarranted diagnoses. In other words, the system is not only able to detect, but also able to correctly abstain.

Takeaway. This robustness confirms the value of a deterministic level 1 and a conditional level 2: decisions remain stable when the data is imperfect, which is essential in a real industrial context.

7.3.5 Scalability

Finally, three scenarios of increasing size were run to obtain a first scaling curve of the complete pipeline.

Dataset	Triples	L1 (ms)	L2 (ms)	Total (ms)
DS25 (small)	170	4,505	254	4,759
DS26 (medium)	258	4,366	242	4,608
DS27 (large)	352	5,823	361	6,434

Table 20: Execution times on three increasing sizes of synthetic graphs.

Over this range of sizes, the total diagnosis time remains below seven seconds. The observed growth remains moderate and suggests that, in our experimental setup, the dominant cost is related more to process orchestration and JVM startup than to the volume of triples itself.

Takeaway. At the scale of the scenarios considered, the complete pipeline is compatible with interactive or near-real-time use.

> *The synthetic catalogue shows that the value of the MAS does not lie only in its accuracy on a few hand-picked cases: it combines accuracy, robustness and short response times on a set of scenarios built precisely to cover the main risks of overfitting to the test cases.*

7.4 Ablation study and comparison with LLM detector and diagnoser

7.4.1 General principle of the comparisons with the LLM

The following experiments aim to situate the added value of the MAS with respect to an approach more directly based on a large language model. Rather than abstractly opposing "MAS" and "LLM", we consider three forms of progressive substitution: letting the LLM directly synthesize a report from the graph, asking it to play the role of a level 1 detector, then asking it to classify the level 2 diagnoses. Each comparison thus targets a specific role in the architecture.

7.4.2 Ablation of the MAS layer for narrative synthesis

This subsection answers question 3 of § 7.1.3: what added value does the MAS bring to a synthesis LLM?

The first experiment compares three narrative synthesis conditions on five representative scenarios: (A) graph only, (B) graph enriched with the L1 outputs, and (C) graph enriched with the L1 and L2 outputs. The model

used is **Mistral-7B-Instruct-v0.3** in float16 on GPU, with three runs per condition. The evaluated metrics are defined as follows:

- **Conformity** — score in $\{0, \frac{1}{3}, \frac{2}{3}, 1\}$ equal to the fraction of three reference elements found in the answer: the expected anchor entity, the expected severity level, and at least one diagnostic keyword defined for the scenario.
- **Hallucination** — proportion of NORIA identifier-like tokens (uppercase strings with underscores or digits) present in the answer but absent from the Turtle graph given as input.
- **Severity** — binary indicator (0 or 1) signaling whether the expected severity level (critical, high, etc.) is mentioned in the answer.
- **Consistency** — average Jaccard similarity over all pairs of runs for the same (scenario, condition), computed on word sets (lowercased, deduplicated tokens).

Condition	Conformity ↑	Hallucination ↓	Severity ↑	Consistency ↑
A — graph only	0.311	**0.000**	0.733	0.328
B — graph + L1	0.533	0.267	0.467	0.433
C — graph + L1 + L2	**0.778**	0.133	**0.733**	**0.420**

Table 21: Average results of the GPU ablation study (5 scenarios × 3 runs, `Mistral-7B-Instruct-v0.3` float16, NVIDIA RTX 4000 Ada, `jaguar.polytechnique.fr`).

The results show a clear gradient A < B < C in conformity, which rises from 0.311 to 0.533 and then 0.778. The most important point, however, is qualitative: the L1 outputs alone improve conformity, but also introduce more hallucinations and a poorer calibration of severity. Level 2 then plays a decisive structuring role: it turns a set of still scattered local signals into correlated hypotheses that are easier for a generative model to use.

Takeaway. The MAS does not help the LLM simply by giving it more information; it helps by giving it information that is already organized.

7.4.3 Replacing level 1 with an LLM detector

This subsection addresses the first part of question 4 of § 7.1.3: can the LLM replace the MAS as a detector?

The second comparison consists of asking the LLM to take on the role of a level 1 detector itself, from the `noria-0.2` graph alone. Two prompt strategies are compared: a guided strategy, which spells out the types of anomalies to look for, and an open strategy, which lets the model freely propose anomalies.

Mode	Strategy	LLM detections	MAS entities	Recall	Precision	Time (s)
A priori	Guided	11	33	0.061	0.182	187.7
A priori	Open	5	33	0.000	0.000	27.8
A posteriori	Guided	4	19	**0.211**	**1.000**	22.8
A posteriori	Open	5	19	0.158	0.750	24.4

Table 22: Performance of the LLM alone as an anomaly detector on `noria-0.2`.

The results are unambiguous. An LLM can identify a few salient anomalies, especially when they are supported by positive evidence present in the graph, but it is not a credible replacement for the deterministic detectors. Recall remains very low, and the failure is particularly marked in a priori mode, where anomalies are often given by the absence of a relation, an interface or coverage.

This baseline shows that the LLM is a poor substitute for level 1: its limitation is not only quantitative, it is structural. Where a SPARQL agent explicitly checks a constraint over the whole graph, the LLM remains a probabilistic reader, unable to guarantee the exhaustiveness of its scan.

7.4.4 Replacing level 2 with an LLM classifier

This subsection addresses the second part of question 4 of § 7.1.3: can the LLM replace the MAS as a diagnoser?

The last comparison concerns a task closer to the core of level 2: explicitly naming, among the twelve implemented diagnosers, those that should trigger on a given scenario. In all conditions, the list of the twelve diagnoser names is given explicitly in the prompt: this is therefore a closed classification, not free generation. Three LLM conditions follow the previous scheme (graph only, graph+L1, graph+L1+L2) and are compared with the MAS reference on the 36 (mode, dataset) pairs of the catalogue for which at least one diagnosis is expected.

Condition	Precision	Recall	F1	TP / FP / FN
A — LLM, graph only	0.111	0.579	0.088	4 / 32 / 20
B — LLM, graph + L1	0.197	0.639	0.191	7 / 35 / 17
C — LLM, graph + L1 + L2	0.426	0.685	0.391	9 / 22 / 15
MAS (reference)	**0.537**	**0.778**	**0.540**	13 / 20 / 11

Table 23: Classification of the L2 diagnosers: LLM (3 conditions) vs MAS.

	A priori (16 pairs)		A posteriori (20 pairs)	
Condition	**F1**	**TP/FP/FN**	**F1**	**TP/FP/FN**
A — LLM graph only	0.104	2/14/9	0.075	2/18/11
B — LLM graph + L1	0.148	5/19/6	0.225	2/16/11
C — LLM graph + L1 + L2	**0.244**	5/13/6	0.508	4/9/9
MAS (reference)	0.196	5/18/6	**0.815**	8/2/5

Table 24: L2 classification F1 by mode.

The main result is that the MAS remains globally superior to the LLM, even in the LLM's best input condition. The overall F1 rises from 0.088 to 0.191 and then 0.391 as the model's input is enriched, which confirms that the layers of the MAS structure information in a way that is useful for the LLM. Nevertheless, the MAS reference reaches 0.540, with a very clear advantage in a posteriori mode, where it climbs to 0.815 against 0.508 for the best LLM. The slight advantage observed for the LLM in a priori mode should be interpreted as the symptom of a localized weakness of some MAS diagnosers, and not as a structural superiority of the generative approach.

All the comparisons with the LLM converge on the same conclusion: the main contribution of the MAS is not to add a decorative symbolic layer around a generative model, but to provide the exhaustiveness, correlation and logical discipline that it lacks. The LLM excels at rephrasing and local reasoning; the MAS remains superior whenever a graph must be covered systematically and a diagnostic trigger must be justified.

8 Conclusion

8.1 Answer to the research question

This PSC addressed the following question, raised from Section 1.1 onwards: *how can we ensure the synergy of a multi-agent system dedicated to anomaly detection in ICT networks, so as to maximize explainability while containing complexity?*

To answer it, we proposed three architectural contributions:

- A **three-level architecture with explicitly separated levels** (deterministic detection L1, conditional reasoning L2, narrative synthesis L3) that decouples responsibilities: each level is developed, tested and evaluated independently of the others, and the *decision / communication* separation guarantees that the LLM never makes a diagnostic decision.
- An **orthogonal multi-family detection model**: 54 agents organized into four NORIA-O families (structural, dynamic, functional, procedural), each specialized in one facet of the network and each operating in a priori or a posteriori mode depending on the operational context.
- A **cross-family correlation framework**: 12 level 2 diagnosers that aggregate the workers' local detections into consolidated diagnoses through convergence of evidence on an anchor entity, quantified by reliability, severity and priority scores.

8.2 Results and empirical answers to the research question

The whole system — 54 L1 agents, 12 L2 diagnosers, 1 L3 narrator — was integrated and validated on the noria-0.2 reference dataset and a catalogue of 27 synthetic datasets covering varied scenarios: base cases, multiple diagnoses, missing data, noise, and load scaling.

The experimental validation provides five concrete answers to the research question.

Synergy is demonstrated and quantified. The four families are non-redundant on noria-0.2 (Section 7.2.2): each detects entities that no other sees. The ablation study on narrative synthesis (Section 7.4.2) shows that moving from the LLM alone (condition A) to LLM + L1 + L2 (condition C) improves diagnostic conformity by **+150 %** (0.311 $\rightarrow$ 0.778): each layer contributes measurably and independently. On the synthetic catalogue, the MAS reaches an a posteriori macro-F1 of **0.815** and an MRR of **0.833** on multi-diagnosis cases.

Explainability is preserved by construction. Each diagnosis is fully traceable from the LLM narrative report down to the RDF triples through the intermediate JSON files. The workers' SPARQL queries are readable and auditable, the triggering conditions of the L2 diagnosers are declarative, and the LLM only rephrases what the deterministic layers have already established. This property, absent from graph-embedding approaches (Knowledge Graph Embeddings) where the decision is a black box, is intrinsic to the chosen architecture.

Complexity is contained and parallelism is an emergent property. The JaCaMo architecture produces a speedup of 12.6$\times$ (a priori) and 18.7$\times$ (a posteriori) without any modification of the individual agents (Section 7, qualitative validation on noria-0.2): parallelism emerges from the coexistence of agents in the same workspace, without explicit coordination. Response time remains under 7 seconds on graphs of 170 to 352 triples. Extensibility comes at a constant marginal cost: adding the 55th agent amounts to writing a Python script and one line in the JCM.

Reasoning by absence is a structural limitation of LLMs, not of the MAS. The subsection on replacing level 1 with an LLM detector (Section 7.4.3) shows that an LLM alone reaches a recall of 0.211 at best (guided a posteriori) and 0.000 on a priori patterns. SPARQL agents guarantee a recall of 1.00 on their domain by construction: exhaustiveness is structural rather than probabilistic, which directly meets the explainability constraint of the ICT domain.

The LLM vs MAS comparison on L2 classification confirms the MAS's own value. On 36 test pairs, the subsection on replacing level 2 with an LLM classifier (Section 7.4.4) shows that the MAS reaches F1 = 0.815 in a posteriori mode against F1 = 0.508 for the best LLM (condition C with full L1+L2 context): the MAS is not a mere "context generator" for the LLM, it is a better classifier on patterns based on positive evidence in the graph.

8.3 Discussion, limitations and perspectives

The results obtained validate the central hypothesis of the project. The *bottom-up* approach — first building a working system rather than abstractly optimizing an objective function — produced a solid empirical basis: measured times, diagnoses validated on 27 datasets, and an architecture whose properties (parallelism, non-redundancy, robustness, scalability) emerged from the design without requiring after-the-fact optimization. We observed that the non-redundancy between NORIA-O families is strongly supported by the semantic structure of the ontology itself, and not only by an additional engineering choice.

These positive results come with identified limitations. First, the `procedural_unreadiness` diagnoser is overconfident (15 FP for 1 TP on the 16 a priori pairs): its overly broad SPARQL conditions flag every resource without an operation plan, even in clean graphs, lowering the a priori macro-F1 to 0.196. Second, 5 of the 12 diagnosers produced no TP on the current synthetic catalogue: their use cases are not yet covered by the 27 datasets. Third, level 3 (Llama 3.2:3b on CPU) accounts for 94.5 % of the end-to-end time in our reference local configuration: sequential CPU inference is a bottleneck not addressed in this project. Fourth, the Moise organization remained conceptual: the roles (Detector, Diagnostician, Narrator) are not formalized as a Moise specification, which prevents using organizational norms to constrain or prioritize the agents' behavior.

These limitations outline the natural perspectives for future work:

- **Refine the a priori diagnosers.** Tighten the SPARQL conditions of `procedural_unreadiness` (add a constraint on severity or resource type) and calibrate the 5 diagnosers without TP by building datasets covering their specific scenarios.
- **Complementary probabilistic agents.** Integrate agents based on Knowledge Graph Embeddings for cases where the evidence is incomplete or the graph is noisy. These agents would coexist with the deterministic workers: the L2 correlation framework is agnostic to the type of L1 agent.
- **Moise formalization and objective function.** Define the system's roles, missions and norms through an explicit Moise specification, then implement the objective function (envisaged since September 2025) as an organizational utility metric driving the prioritization of the active diagnosers.
- **Optimization of level 3.** Serve the LLM on a dedicated GPU (`jaguar.polytechnique.fr` with Mistral-7B-Instruct-v0.3 float16 is already available and validated in the comparisons of Section 7.4), or move the narrative to an asynchronous task so as not to block the diagnosis.
- **Dynamic enrichment of the graph.** Systematize the re-injection of inferred diagnoses into the graph so that future agents benefit from the history of past diagnoses: a dynamic, self-annotated knowledge graph.

9 References

[1] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser, et al. "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI". In: *Information Fusion* 58 (2020), pp. 82–115. doi: 10.1016/j.inffus.2019.12.012.

[2] R. H. Bordini, J. F. Hübner, and M. Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason*. Wiley Series in Agent Technology. John Wiley & Sons, 2007. isbn: 978-0-470-02900-8.

[3] A. Hogan, E. Blomqvist, M. Cochez, et al. *Knowledge Graphs*. Synthesis Lectures on Data, Semantics, and Knowledge. Springer, 2021. doi: 10.1007/978-3-031-01918-0.

[4] A. Hussain, A. A. Bui, and Y. K. Suh. "Multi-agent systems and their applications". In: *International Journal of Computing and Digital Systems* 6.3 (2017), pp. 133–140.

[5] JaCaMo Project. *JaCaMo: A Multi-Agent Oriented Programming Platform*. https://jacamo-lang.github.io/jacamo/. Consulté en avril 2026. 2024.

[6] M. Z. Muchika. "Knowledge Management Framework for Autonomous Digital Twins". PhD thesis. Aalto University, 2023.

[7] Orange OpenSource. *NORIA Ontology Dataset (noria-0.2)*. GitHub repository. 2023. url: https://github.com/Orange-OpenSource/noria-ontology/tree/master/dataset/noria-0.2.

[8] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu. "Unifying Large Language Models and Knowledge Graphs: A Roadmap". In: *IEEE Transactions on Knowledge and Data Engineering* 36.7 (2024), pp. 3580–3599. doi: 10.1109/TKDE.2024.3352100.

[9] A. S. Rao and M. P. Georgeff. "BDI Agents: From Theory to Practice". In: *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*. San Francisco, CA, USA, 1995, pp. 312–319.

[10] C. Rudin. "Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead". In: *Nature Machine Intelligence* 1.5 (2019), pp. 206–215. doi: 10.1038/s42256-019-0048-x.

[11] J. Sun, C. Xu, L. Tang, et al. "Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph". In: *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. 2024. url: https://openreview.net/forum?id=nnVO1PvbTv.

[12] L. Tailhardat. "Anomaly Detection using Knowledge Graphs and Synergistic Reasoning: Application to Network Management and Cyber Security". Thèse de doctorat. Sorbonne Université, 2024. url: https://thesjaces.hal.science/THESES-SU/tel-04831512v1.

[13] L. Tailhardat, Y. Chabot, and R. Troncy. "NORIA-O: An Ontology for Anomaly Detection and Incident Management in ICT Systems". In: *Semantic Web Journal* (2023). url: https://www.semantic-web-journal.net/system/files/swj3334.pdf.

[14] W. van der Aalst. *Process Mining: Data Science in Action*. 2nd ed. Springer, 2016. doi: 10.1007/978-3-662-49851-4.

[15] Q. Wang, Z. Mao, B. Wang, and L. Guo. "Knowledge Graph Embedding: A Survey of Approaches and Applications". In: *IEEE Transactions on Knowledge and Data Engineering* 29.12 (2017), pp. 2724–2743. doi: 10.1109/TKDE.2017.2754499.

[16] World Wide Web Consortium. *Semantic Web Standards*. https://www.w3.org/standards/semanticweb/. Consulté en avril 2026. 2023.

[17] Q. Wu, G. Bansal, J. Zhang, et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. arXiv: 2308.08155 [cs.AI]. url: https://arxiv.org/abs/2308.08155.